Dataset Used: kc_house_data.csv

The dataset used is **KC House Data**, which contains housing details along with their corresponding prices.
It includes features such as bedrooms, bathrooms, square footage, floors, condition, grade, and year built.
The target variable is **house price**, which is a continuous value representing the cost of each house.
This dataset is commonly used for **regression problems**, as it involves predicting numerical values.
In this project, both **classification and regression algorithms** are applied to understand different machine learning approaches and compare their performance.

Goal: The main goal of this project is to apply and evaluate multiple machine learning algorithms on the KC housing dataset.
It aims to analyze how different models perform using housing features such as area, bedrooms, and condition.
The project focuses on understanding the importance of data preprocessing and feature scaling in improving model performance.
Another objective is to compare both classification and regression algorithms to identify the most effective approach.
Finally, it helps in determining which type of model is more suitable, concluding that regression is better for predicting continuous values like house prices.

Modules Used:

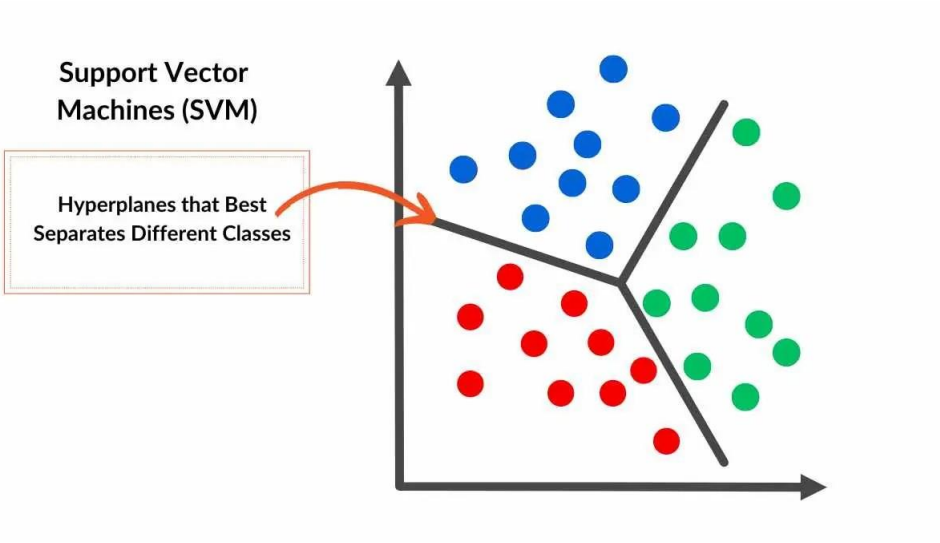
The **primary libraries/modules** required for this project are:

- **Pandas:** For all data manipulation, filtering, aggregation, and feature engineering tasks.
- **NumPy:** Specifically for numerical calculations like yearly growth (`np.diff`).
- **Scikit-Learn:** Scikit-learn is the main machine learning library used in this project. It provides tools for data splitting, preprocessing, model building, and evaluation. All classification algorithms used in this project come from this library.
- **train_test_split:** This module is used to divide the dataset into training and testing sets. It ensures that the model is evaluated on unseen data. In this project, 80% data is used for training and 20% for testing.
- **StandardScaler:** StandardScaler is used for feature scaling. It standardizes features by converting them into a common scale. This improves model performance and prevents bias due to feature range differences.
-
- **Classification Algorithms (from Scikit-learn):**
    - SVC – Used for Support Vector Machine classification
    - LogisticRegression – Used for probability-based classification
    - GaussianNB – Used for probabilistic classification
    - DecisionTreeClassifier – Used for tree-based decisions
    - RandomForestClassifier – Used for ensemble learning
    - KNeighborsClassifier – Used for distance-based classificatio (KNN)
    - GradientBoostingClassifier – Used for sequential ensemble learning
- **Regression Algorithms (Scikit-learn)**
    - LinearRegression – Predicts continuous values
    - Ridge – Regularized linear regression
    - DecisionTreeRegressor – Tree-based regression
    - RandomForestRegressor – Ensemble regression
    - SVR – Support Vector Regression
    - GradientBoostingRegressor – Boosting-based regression
- **accuracy_score:** This module is used to evaluate the model performance. It calculates the accuracy of predictions made by each algorithm. Helps in comparing different models.
- **Evaluation Metrics**
    - accuracy_score – Used for classification model evaluation
    - r2_score – Used for regression model evaluation

Classification Algorithms Implemented in the Project

1. Support Vector Machine (SVM)

Support Vector Machine (SVM) is a supervised learning method that classifies the data by finding the optimal boundary between categories with maximum margin. In housing project, it uses features like bedrooms, area, and condition to group houses into price categories. It works well with high-dimensional and non-linear data using kernels, but is less suitable since housing prices are continuous. Additionally, it can struggle with overlapping price ranges and requires careful parameter tuning, making it computationally intensive for large datasets.



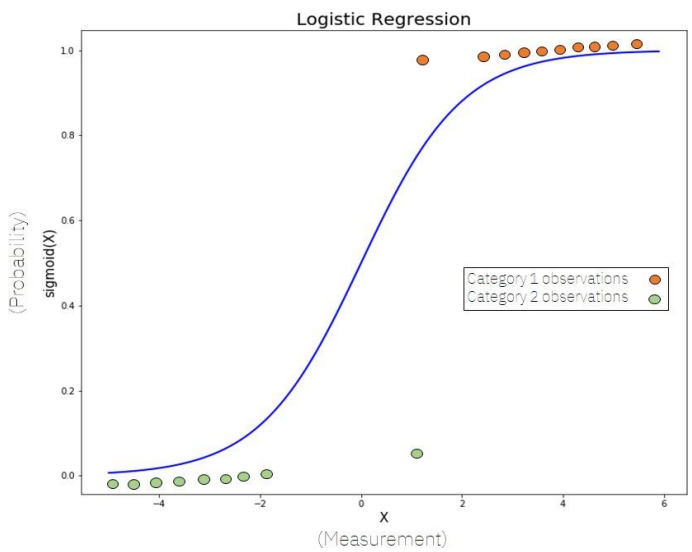
SVM Classifier

Limitations

- SVM is computationally expensive on large datasets, especially with non-linear kernels, as training time grows rapidly with more samples.
- Choosing the right kernel and tuning hyperparameters (like C and gamma) is difficult and often requires extensive experimentation.
- It performs poorly with noisy data or overlapping classes because it tries to create a clear margin between categories.
- SVM is less interpretable and sensitive to feature scaling, making it harder to understand decisions and requiring proper normalization for good performance.

2. Logistic Regression

Logistic Regression is a supervised classification algorithm that predicts probabilities and assigns categories based on a threshold. It is simple, fast, and works well when there is a linear relationship between features and output. However, it may oversimplify complex housing data with non-linear patterns. It is also sensitive to outliers and not suitable for predicting exact continuous price values.



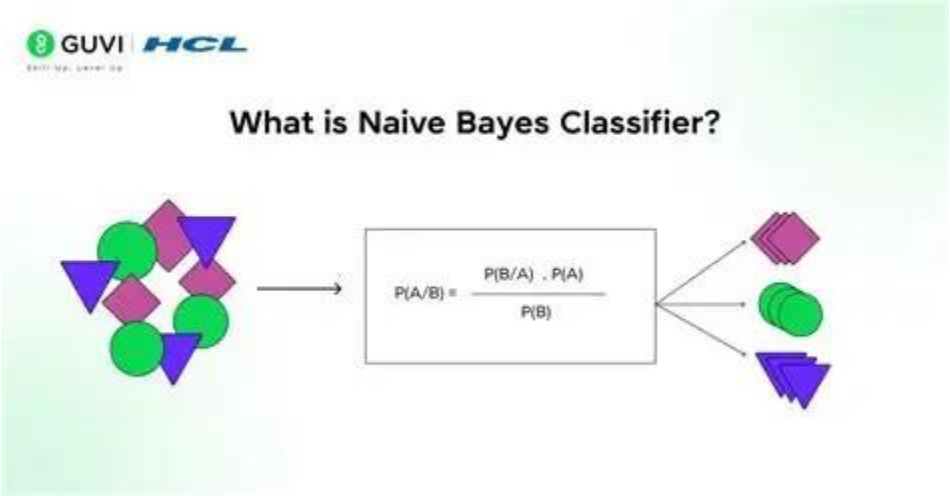
Logistic Regression

Limitations

- Logistic Regression assumes a linear relationship, so it performs poorly on complex, non-linear data.
- It struggles with non-linear decision boundaries unless additional feature transformations are applied.
- The model is sensitive to outliers and multicollinearity, which can distort predictions and reduce stability.
- It may underfit complex datasets, leading to lower accuracy compared to more advanced models.

3. Naive Bayes (GaussianNB)

Naive Bayes is a probabilistic classifier that predicts house price categories assuming features are independent and normally distributed (in GaussianNB). It is fast and works well with high-dimensional data, as seen in the graph with clear but simple decision boundaries. However, the graph may show misclassifications due to correlated housing features, violating independence assumptions. This leads to oversimplified predictions and reduced accuracy.



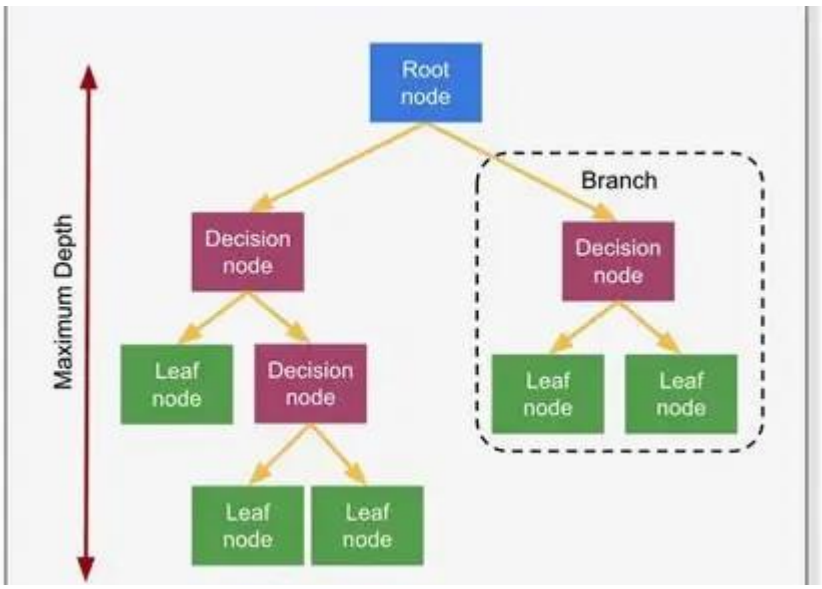
Naive Bayes (GaussianNB)

Limitations

- Naive Bayes assumes feature independence, which is often unrealistic and reduces accuracy when features are correlated.
- GaussianNB assumes normal distribution of data, which may not fit real-world, skewed datasets.
- It may achieve lower accuracy when complex feature interactions are important.
- Although fast and efficient, it can oversimplify patterns and struggle with imbalanced data.

4. Decision Tree Classifier

Decision Tree is a supervised algorithm that splits data into branches based on feature values to make decisions, capturing non-linear patterns in housing data. It is easy to understand and can handle both numerical and categorical features. However, it is prone to overfitting and sensitive to small data changes. Deep trees can become complex and may not generalize well to new data.



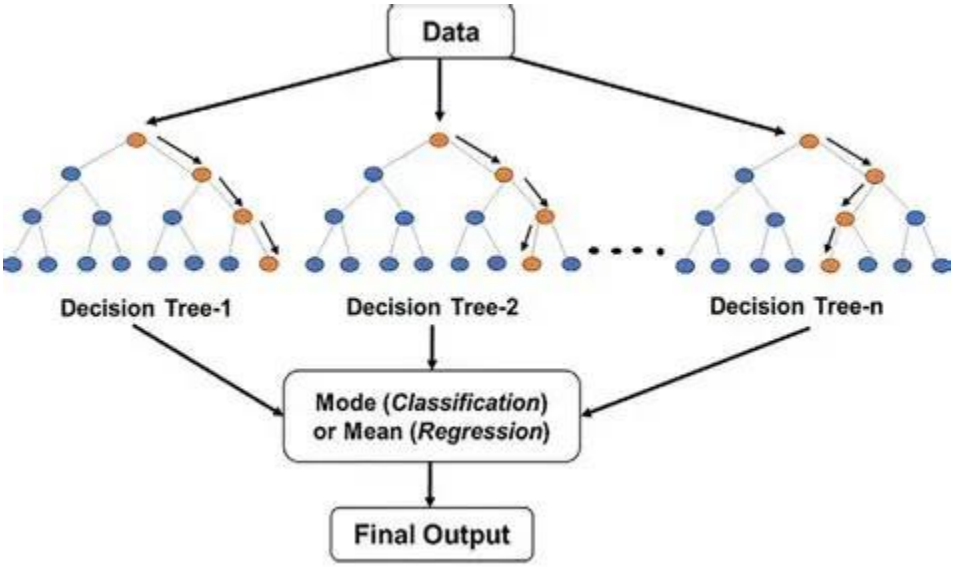
Decision Tree Classifier

Limitations

- Decision Trees are prone to overfitting, as deep trees may capture noise instead of general patterns.
- They have high variance, meaning small data changes can lead to very different tree structures.
- They may not generalize well to new data without pruning or limiting depth.
- With imbalanced or high-dimensional data, trees can become biased and overly complex, reducing interpretability.

5. Random Forest Classifier

Random Forest is an ensemble algorithm that combines multiple decision trees trained on different subsets of data and features, using voting for final predictions. It improves accuracy and reduces overfitting while capturing complex relationships in housing data. It performs well on large datasets but requires higher computational power. However, it is less interpretable and not ideal for predicting exact continuous price values.



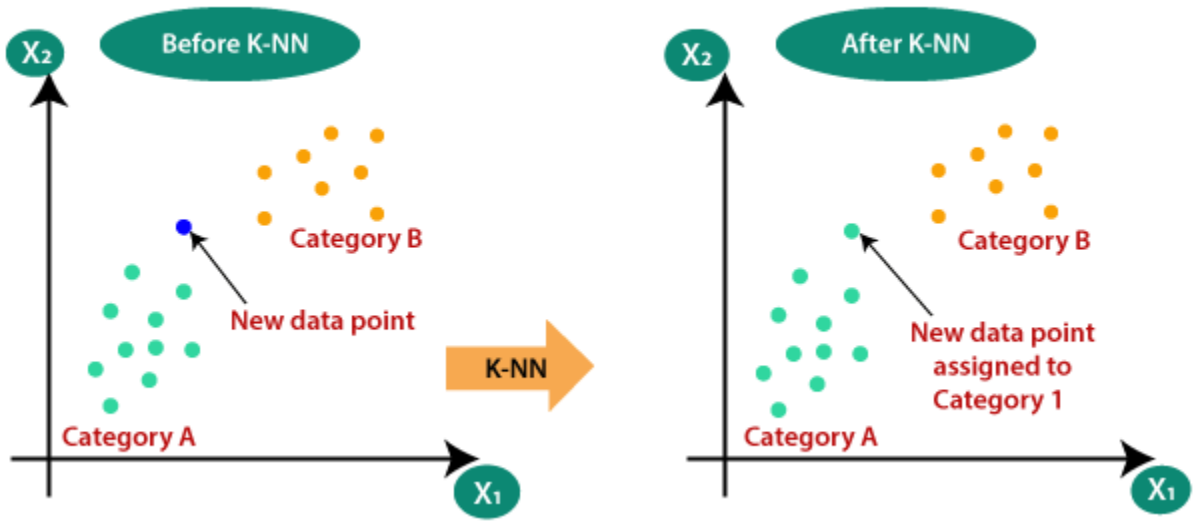
Random Forest Algorithm

Limitations

- Random Forest increases computational cost, as it builds multiple trees, requiring more time and memory.
- Training can be slower, especially with many trees and large datasets.
- It is less interpretable than a single Decision Tree, making predictions harder to explain.
- It may not perform well on small datasets and requires careful hyperparameter tuning to avoid unnecessary complexity.

K-Nearest Neighbors (KNN)

K-Nearest Neighbours (KNN) is a distance-based algorithm that classifies a house based on the majority class of its nearest neighbours using features like area and rooms. It is simple and effective when similar data points are close in the dataset. However, it becomes slow with large datasets and requires feature scaling for good performance. It is also sensitive to outliers and performs poorly in high-dimensional data.



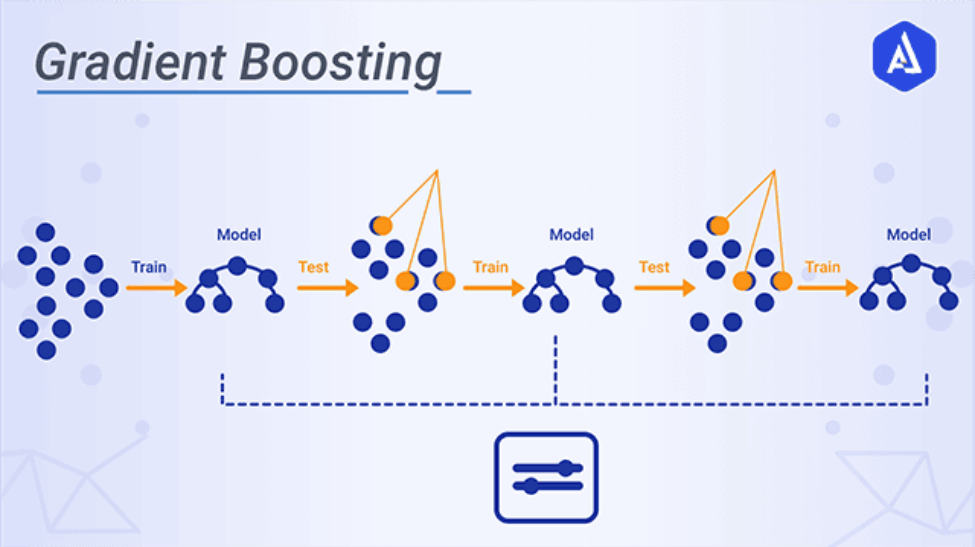
Explaining KNN

Limitations

- KNN has high computational cost because it calculates distances to all training points during prediction, making it slow for large datasets.
- Choosing the right value of K is difficult, as small K can cause overfitting and large K can lead to underfitting.
- It suffers from the curse of dimensionality, where performance drops as the number of features increases.
- It is sensitive to feature scaling, outliers, and requires high memory since it stores the entire training dataset.

Gradient Boosting Classifier

Gradient Boosting is an ensemble method that builds models sequentially, where each new model corrects the errors of the previous ones using weak learners like decision trees. It captures complex relationships in housing data and delivers high accuracy for structured datasets. However, it is computationally slow due to its sequential training process and requires careful hyperparameter tuning. It is also sensitive to noise and may overfit if too many trees are added.



Gradient Boosting

Limitations

Despite its high performance, Gradient Boosting has several drawbacks:

- **High Computational Complexity**
  Training is slow because models are built sequentially. Each step depends on the previous one, making it time-consuming.
- **Requires Careful Hyperparameter Tuning**
  Parameters like:
  - Learning rate
  - Number of trees
  - Tree depth

    must be tuned properly. Poor tuning can reduce performance.
- **Risk of Overfitting**
  If too many trees are added or the learning rate is too high, the model may memorize training data instead of generalizing.
- **Sensitive to Noise and Outliers**
  Since each model tries to correct previous errors, noisy data can get amplified, affecting accuracy.
- **Less Interpretability**
  The model is complex and difficult to interpret compared to simpler models like Decision Trees.
- **Sequential Nature (No Parallelization)**
  Unlike Random Forest, Gradient Boosting cannot be easily parallelized, increasing training time.

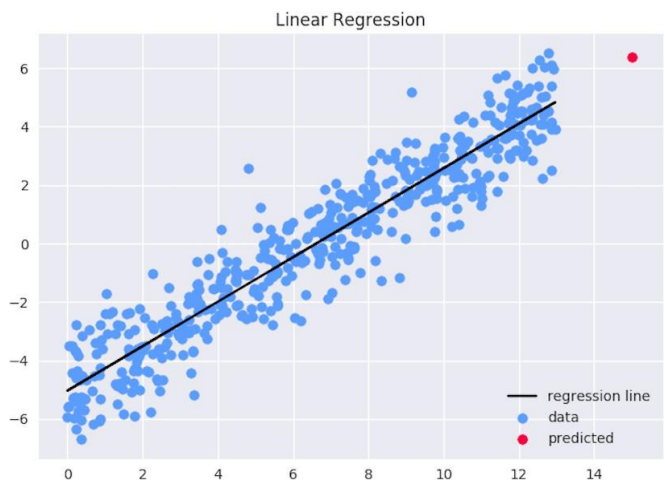
Conclusion

Gradient Boosting delivers strong predictive performance but at the cost of higher complexity, longer training time, and careful tuning requirements.

Regression Algorithms Implemented in the Project

Linear Regression

Linear Regression is a basic regression algorithm that models the relationship between input features and a continuous target variable. In this project, it predicts house prices based on features such as bedrooms, square footage, and condition. It assumes a linear relationship between features and price. The model is simple, fast, and easy to interpret, making it useful as a baseline model. However, housing data often contains non-linear relationships that Linear Regression cannot capture effectively. It may underfit complex data. It is also sensitive to outliers, which can affect prediction accuracy.



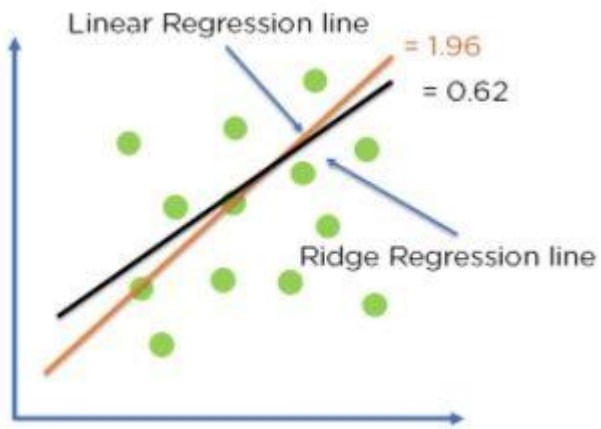
Linear Regression

Limitations

- Linear Regression assumes a linear relationship, which may not hold true for housing data.
- It is sensitive to outliers and can give inaccurate predictions.
- It cannot capture complex feature interactions.
- It may underfit when data is non-linear.

Ridge Regression

Ridge Regression is an improved version of Linear Regression that adds regularization to reduce overfitting. It penalizes large coefficients and helps stabilize the model. In this project, it is used to predict house prices while handling correlated features like sqft_living and sqft_lot. It improves generalization compared to Linear Regression. It is useful when multiple features influence price. However, it still assumes a linear relationship and may not capture complex patterns in housing data.



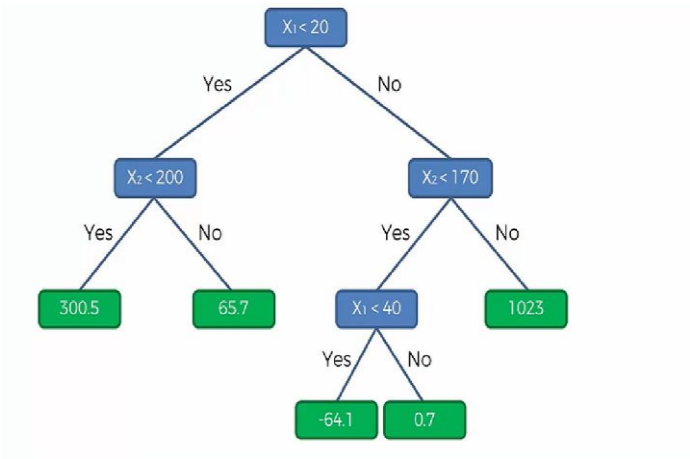
Linear Regression vs Ridge Regression

Limitations

- It cannot handle strong non-linear relationships.
- It does not perform feature selection (keeps all features).
- Performance depends on tuning the regularization parameter.
- May still underfit complex housing data.

Decision Tree Regressor

Decision Tree Regressor is a tree-based algorithm that predicts continuous values by splitting data into regions. It creates rules based on features like area and condition to estimate house prices. In this project, it captures non-linear relationships effectively. It is easy to interpret and visualize. It works well with structured data. However, it is prone to overfitting and may create overly complex trees. Small changes in data can lead to different results.



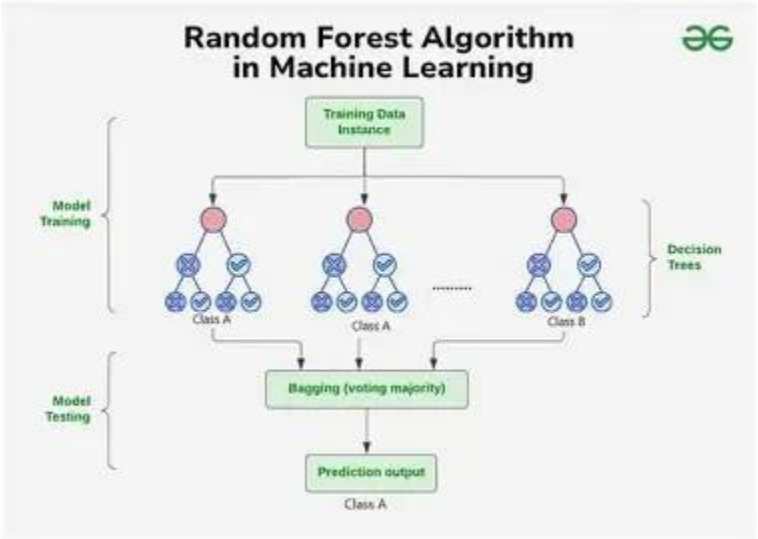
Decision Tree Regression

Limitations

- Highly prone to overfitting if not controlled.
- Sensitive to small changes in data.
- May not generalize well to unseen data.
- Requires pruning or tuning for better performance.

Random Forest Regressor

Random Forest Regressor is an ensemble method that combines multiple decision trees to improve prediction accuracy. Each tree is trained on different subsets of data and features. In this project, it predicts house prices more accurately by capturing complex relationships. It reduces overfitting compared to a single decision tree. It performs well on large datasets like KC housing data. However, it requires more computational resources and is less interpretable.



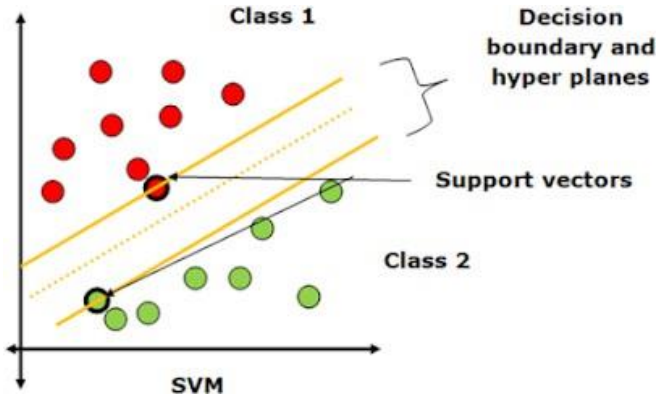
Random Forest Algorithm

Limitations

- Requires more memory and computation.
- Less interpretable compared to simple models.
- Training can be slower with many trees.
- Needs parameter tuning for best performance.

Support Vector Regressor (SVR)

Support Vector Regressor (SVR) is the regression version of SVM that predicts continuous values. It tries to fit the best line within a margin of tolerance. In this project, it models relationships between housing features and price. It works well for smaller datasets and complex patterns. It uses kernel functions to handle non-linear relationships. However, it is computationally expensive and not suitable for very large datasets.



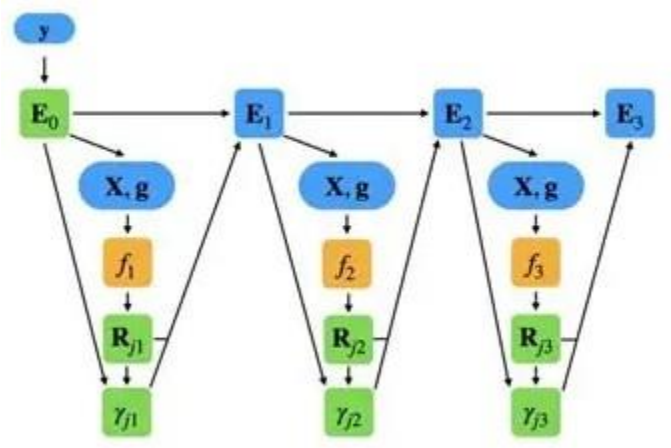
Support Vector Regressor

Limitations

- Slow for large datasets.
- Requires careful parameter tuning.
- Sensitive to feature scaling.
- Performance decreases with large and noisy data.

Gradient Boosting Regressor

Gradient Boosting Regressor is an ensemble algorithm that builds models sequentially to reduce prediction errors. Each new model improves upon the previous one. In this project, it predicts house prices by capturing complex feature interactions. It provides high accuracy and strong performance. It is widely used in real-world applications. However, it requires careful tuning and is computationally expensive.



Gradient Boosting Regressor

Limitations

- Training is slow due to sequential learning.
- Sensitive to noise and outliers.
- Requires careful hyperparameter tuning.
- Can overfit if too many trees are used.

Conclusion

In this section, regression algorithms were applied to predict house prices using the KC housing dataset.

Models like Linear Regression provided a simple baseline, while ensemble methods such as Random Forest and Gradient Boosting delivered better performance.

Tree-based models handled non-linear relationships effectively compared to linear models.

Among all, ensemble regressors showed higher accuracy due to their ability to capture complex patterns.

Overall, regression algorithms proved more suitable for this dataset as they predict continuous values like house prices more accurately.

Final Conclusion

This project demonstrates a complete machine learning workflow, including data preprocessing, feature scaling, model training, and evaluation using the KC housing dataset.

Both classification and regression algorithms were applied to understand and compare their performance on real-world data.

Classification models helped in learning different approaches, while regression models provided more accurate predictions for house prices.

Among all models, ensemble methods like Random Forest and Gradient Boosting showed better performance due to their ability to capture complex relationships.

Since house price is a continuous variable, regression algorithms are more suitable and effective for practical prediction tasks.

What we learned:

Through this project, we gained practical knowledge of applying both classification and regression algorithms to real-world housing data.
We learned how to load, preprocess, and scale data using tools like Pandas and StandardScaler.
We understood how different models work and how their performance varies based on data characteristics.
We also learned to train models, make predictions, and evaluate results using both accuracy (classification) and R^2 score (regression).
Additionally, we realized the importance of choosing the right approach, as regression is more suitable for predicting continuous values like house prices.